Adaptive Scheduling for Load-Balanced Parallel N-Body Simulation: A Hybrid OpenMP–MPI–CUDA Approach

Tanish Hemant Kuvar
Reg. No. 240962540
AIML-A
Manipal Institute of Technology

M. Vishal Kumar
Reg. No. 240962546
AIML-A
Manipal Institute of Technology

Samyak Jain
Reg. No. 240962638
AIML-A
Manipal Institute of Technology

***Abstract*—Gravitational N-body systems cluster as they evolve, so an equal division of bodies among threads stops being an equal division of work. We present a hybrid MPI–OpenMP–CUDA Barnes–Hut simulator in which bodies are Morton-ordered, so that every rank and every thread owns a compact region of space, and in which a lightweight run-time controller selects the OpenMP loop schedule from measured per-thread loop times. The controller switches between static and dynamic scheduling with hysteresis through the standard omp_set_schedule() interface; to avoid oscillation it predicts the imbalance a static schedule would have while dynamic scheduling is active. On a laptop processor with eight performance cores the simulator reaches a speed-up of 5.6–6.4× at 8 threads for N = 65,536 and produces bit-identical results across all thread counts, rank counts and schedules. The controller stays within 2.3 % of static scheduling on a balanced input and is faster than static in all nine clustered configurations, by 1.5–16.6 %; the built-in guided schedule, however, matched or exceeded it in 9 of 12 cases. A tiled CUDA direct-summation kernel is 88× faster than 8 CPU threads in single precision.**

***Index Terms*—N-body simulation, Barnes–Hut, MPI, OpenMP, CUDA, load balancing, adaptive loop scheduling.**

I. Introduction

An N-body simulation advances N mutually interacting bodies one timestep at a time. Under softened Newtonian gravity the acceleration of body i is

$$a_i = G \sum_{j \neq i} m_j \, (x_j - x_i) \, / \, (|x_j - x_i|^2 + \varepsilon^2)^{3/2} \qquad (1)$$

where ε is a softening length. Direct summation needs $N(N-1)/2$ interactions per step, about 2.1×10^9 at N = 65,536. The Barnes–Hut method [1] reduces the cost to O(N log N) by treating distant groups of bodies as single masses, but large systems still need parallel hardware: MPI across nodes, OpenMP across cores and CUDA on GPUs.

The cost of a body in Barnes–Hut grows with the density around it, because a body in a dense region opens more tree cells. As a system clusters, the threads that own dense regions do far more work than those that own sparse ones. A fixed static schedule then leaves threads idle, while a fixed dynamic schedule pays scheduling overhead even when the load is uniform. Because clustering develops during a run, the best schedule changes within a single run, which makes run-time switching attractive.

This report makes four contributions:

1) a hybrid MPI–OpenMP–CUDA Barnes–Hut simulator with a Morton-ordered domain decomposition whose final state is bit-identical across schedules, thread counts and rank counts;
2) a run-time schedule controller built only on omp_set_schedule(), including a correction to the synopsis algorithm that predicts static thread times while dynamic scheduling is active;
3) an evaluation on uniform and clustered inputs (four inputs, N up to 65,536, 1–8 threads, 1–4 ranks) with measurement controls for a hybrid, thermally limited laptop;
4) CUDA kernels compiled at run time with NVRTC: a tiled direct-summation kernel and a Barnes–Hut near-field hybrid.

Section II reviews related work, Section III states the problem, Section IV describes the design, Section V the methodology, Section VI the results, and Sections VII–IX discuss limitations, future work and conclusions.

II. Related Work

A. Parallel N-Body Methods

Singh et al. [2] first showed that tree codes need cost-aware partitioning, and the tiled CUDA kernel of Nyland et al. [3] and the all-GPU Barnes–Hut of Burtscher and Pingali [4] remain the standard GPU baselines. Keller et al. [5] built Cornerstone, which constructs distributed octrees entirely on GPUs and ran a Barnes–Hut code with up to 8 trillion particles. Cazalbou et al. [6] proposed hybrid MPI–GPU algorithms for distributed octree construction, and Nagarajan et al. [7] reformulated Barnes–Hut as a ray-tracing problem for GPU ray-tracing cores.

B. Automatic OpenMP Loop Scheduling

LB4OMP [8] added over a dozen dynamic and adaptive loop scheduling techniques to the LLVM OpenMP runtime. Auto4OMP [9] selects the algorithm and chunk size during execution, partly using a load-imbalance threshold, and Ciorba et al. [10] summarised this automated selection. Müller Korndörfer et al. [11] compared expert-based and reinforcement-learning selection, found run-time selection beneficial, and named CPU–GPU and cross-node selection as future work. Da Silva et al. [12] auto-tuned the chunk size of dynamic scheduling for full waveform inversion.

C. Algorithm Selection in Particle Simulations

AutoPas [13] selects among short-range particle algorithms and parallelisations at run time; Newcome et al. extended it to multi-site molecular dynamics [14] and compared selection strategies [15].

Automated schedule selection is mature at the node level, but [8]–[11] rely on custom runtimes, and recent N-body work [5]–[7] targets force computation rather than thread scheduling. We test a portable controller that uses only the standard OpenMP interface, on a clustering gravitational workload inside a full MPI–OpenMP–CUDA pipeline.

III. Problem Statement and Objectives

In parallel N-body simulation an equal division of bodies does not give an equal division of work once bodies cluster, and no single OpenMP schedule is optimal for a whole run. We ask whether a run-time controller that measures per-thread load and switches the OpenMP schedule mid-simulation can reduce total execution time compared with fixed static, dynamic and guided schedules, in a hybrid MPI–OpenMP–CUDA implementation. The objectives were: (O1) sequential direct-sum and Barnes–Hut baselines with energy and momentum checks; (O2) OpenMP force computation and a characterisation of static, dynamic and guided schedules; (O3) MPI distribution across ranks and a CUDA force kernel; (O4) the adaptive controller, evaluated on uniform and clustered inputs. All four were implemented and measured.

IV. Design and Implementation

A. Simulation Core

Bodies are advanced with the kick–drift–kick leapfrog integrator ($G = 1$, $\varepsilon = 0.05$, $\Delta t = 0.01$). Forces use Barnes–Hut with opening angle $\theta = 0.5$: a cell of size s at distance d from its centre of mass is used as a single mass if $s/d < \theta$ and the cell does not contain the body itself; leaves hold at most 8 bodies. Every step all bodies are sorted along a Morton (Z-order) curve with a stable radix sort of 63-bit keys (21 bits per axis). Because the order is sorted, every octree cell owns a contiguous index range, so the tree is built top-down by binary search on the next three key bits; single-child chains are collapsed, giving at most $2N-1$ nodes.

B. Hybrid Decomposition

Fig. 1 shows one timestep. Each MPI rank owns a contiguous block of the Morton order. After the first half-kick and drift of its own block, every rank exchanges positions and velocities with MPI_Allgatherv (block sizes may differ), so all ranks hold the full state; each rank then sorts all bodies and builds the full tree, and computes forces only for its own block. Inside a rank the force loop is an OpenMP loop with schedule(runtime), so the schedule can be changed between steps without recompilation. Each thread writes only the accelerations of its own bodies, so the loop has no data races. A per-body work count (tree nodes visited plus interactions) is recorded for the controller.

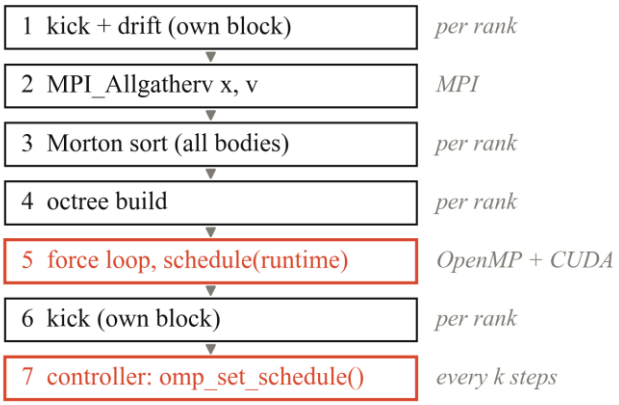


Fig. 1. One timestep on every MPI rank. Highlighted: the OpenMP force loop whose schedule is chosen at run time, and the controller that chooses it.

Morton ordering makes the tree walk cache-friendly, but it also means a static block is a compact region of space. When that region contains a cluster, one thread receives most of the expensive bodies (Fig. 2).

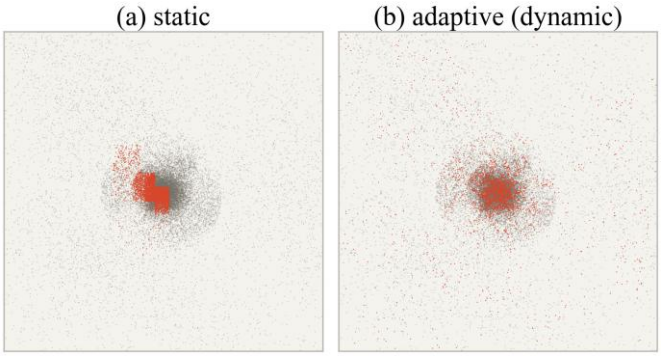


Fig. 2. Cold collapse, N = 32,768, 8 threads, t = 1.50. Highlighted: bodies computed by the busiest thread. (a) Under static scheduling it owns one block of the dense core and takes 3.7× as long as the least-loaded thread ($\lambda = 0.63$). (b) After the controller switches to dynamic its bodies are spread over the cloud.

C. CUDA Kernels

The usual CUDA compiler requires the MSVC host compiler on Windows, which was not available, so the kernels are compiled at run time with NVRTC for the detected device (a native cubin, with a PTX fallback) and launched through the CUDA Driver API; both libraries are loaded dynamically, so the program also runs without a GPU. The direct-summation kernel follows Nyland et al. [3]: each thread computes one body, and a block of 256 threads stages 256 bodies at a time in shared memory. In the hybrid Barnes–Hut mode, CPU threads walk the tree and accumulate the far field; near-field leaves are recorded as index ranges, contiguous thanks to the Morton order and merged when adjacent, and a second kernel evaluates those pairs. Both kernels support single and double precision.

D. Adaptive Controller

Every k steps the controller computes the imbalance of the per-thread loop times t_i, accumulated over the window:

$$\lambda = \max_i t_i \,/\, \mathrm{mean}_i\, t_i - 1 \qquad (2)$$

Two thresholds give hysteresis (Algorithm 1). Under dynamic scheduling, however, the measured t_i are balanced by construction: λ would always fall below the lower threshold, and the controller of the synopsis would switch back to static at every check and oscillate. While dynamic is active, the controller therefore evaluates the thread times that static would produce,

$$T_i = B_i / v_i, \quad v_i = W_i / t_i \tag{3}$$

where B_i is the work in thread i's static block and W_i the work it actually executed. Under static, $W_i = B_i$ and $T_i = t_i$, which is exactly the synopsis metric. The prediction includes both workload clustering and differences in core speed.

Algorithm 1: Adaptive schedule controller (per MPI rank)

```
input: λ_low = 0.05, λ_high = 0.15, k = 5, chunk c = 8; policy ← STATIC
for each timestep s do
  force loop with schedule(runtime); add t_i, W_i, B_i to the window
  if s mod k = 0 then
    λ ← imbalance of t_i (STATIC) or of T_i (DYNAMIC)
    if STATIC and λ > λ_high then policy ← DYNAMIC(c)
    else if DYNAMIC and λ < λ_low then policy ← STATIC
    omp_set_schedule(policy, c); reset window
  second half-kick
```

E. Determinism and Measurement

Each body's force is computed with the same operation sequence whichever thread or rank computes it, and every rank sorts and builds the same tree, so the final state is bit-identical for every schedule, thread count, rank count and controller decision. Per-thread times use QueryPerformanceCounter, because omp_get_wtime() in the MinGW runtime has a resolution of 1 ms. The test CPU mixes performance and efficiency cores; unpinned threads produced λ ≈ 0.3 on a perfectly balanced input, so threads are pinned one per performance core, detected through the Windows EfficiencyClass.

V. Experimental Methodology

TABLE I
Experimental Setup

Item	Configuration
CPU	Intel Core i7-14700HX, 8 P + 12 E cores
GPU	NVIDIA RTX 4060 Laptop GPU, 8 GB
Software	GCC 15.1 -O3 -march=native, MS-MPI 10.1, NVRTC 12.9
Inputs	uniform, Plummer, colliding clusters, cold collapse
Sizes	N = 4,096, 16,384, 65,536; 150 steps
Parallel	1, 2, 4, 8 threads per rank; 1, 2, 4 ranks
Protocol	3 repetitions, shuffled order, median

The uniform input is a sphere in homologous expansion whose density stays uniform; its work imbalance stays between 0.006 and 0.012 throughout, making it a balanced control. The Plummer sphere starts in equilibrium (virial ratio 0.986), the colliding clusters are imbalanced from the start, and the cold collapse is uniform at first and forms a dense core after t ≈ 1. Speed-up and efficiency are

$$S(p) = T(1) / T(p), \quad E(p) = S(p) / p \tag{4}$$

with $T(1)$ the single-thread static run. Long sessions throttled the laptop by up to about 40 %, so later experiments ran behind a thermal gate: before every eight runs a reference configuration had to be within 8 % of its normal time. Throttled measurements were discarded. Because single-core turbo differed between sessions, MPI speed-ups are reported against the single-thread time of the schedule experiment, the conservative baseline.

VI. Results

A. Correctness

TABLE II
Validation Checks (All Passed)

Check	Result
Bit-identical state, 13 configurations	1 checksum
GPU direct vs CPU direct (rms)	1.8×10^{-16} fp64, 1.6×10^{-6} fp32
BH+GPU hybrid vs CPU Barnes–Hut	identical to 7 digits
Barnes–Hut force error, θ = 0.3/0.5/0.7/0.9	0.05/0.25/0.62/1.16 %
Max energy drift, direct / Barnes–Hut	1.1×10^{-7} / 1.4×10^{-5}
Leapfrog order (error ratio, halved Δt)	4.05, 4.02 (ideal 4)
Parallel sort and tree build	identical for 1, 3, 8 threads

B. Imbalance Dynamics

Fig. 3 follows one cold-collapse run at N = 65,536 and 8 threads. The work imbalance of the static blocks stays below 0.05 until t = 1.00 and then rises to 0.64 as the core forms; the imbalance measured under static follows it. The adaptive run switched briefly at t = 0.41 on a timing spike, which the lower threshold reverted after one window, and permanently at t = 1.16. At the last step the measured imbalance was 0.57 under static and 0.04 under adaptive scheduling.

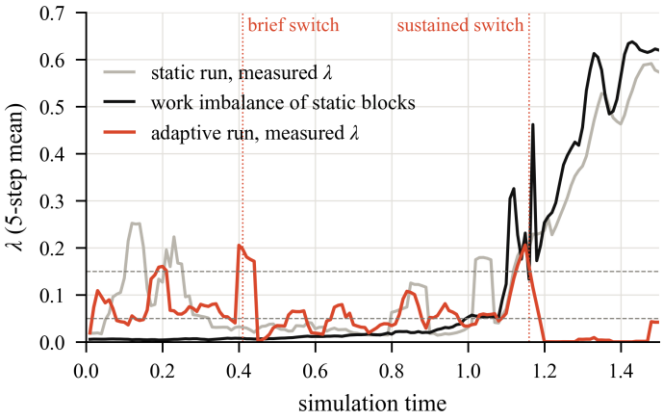


Fig. 3. Imbalance during a cold collapse (N = 65,536, 8 threads, 5-step mean). Dashed lines: the controller thresholds 0.05 and 0.15; dotted lines: the adaptive run's switches to dynamic.

C. Schedule Comparison

Table III lists the speed-up at 8 threads and N = 65,536; efficiency ranges from 70 % to 81 %. Fig. 4 shows the run time saved relative to static for all nine clustered configurations at 8 threads. Every balancing schedule beats static in all of them.

The adaptive controller is faster than static in 9 of 9 cases, by 1.5–16.6 %, and stays within 2.3 % of static on the uniform input, as the synopsis predicted. On the small uniform input (N = 4,096), dynamic scheduling with chunk 8 was 46 % slower than static, while the controller stayed on static. Guided scheduling was the best fixed schedule in 9 of 12 cases and adaptive in 3. Fig. 5 shows that static falls behind only on the clustered input.

TABLE III
SPEED-UP AT 8 THREADS, N = 65,536

Input	Static	Dynamic	Guided	Adaptive
Uniform	6.06	5.73	6.13	5.92
Plummer	5.70	6.26	6.44	6.03
Collide	5.67	6.18	6.39	6.02
Collapse	5.60	5.87	5.95	5.69

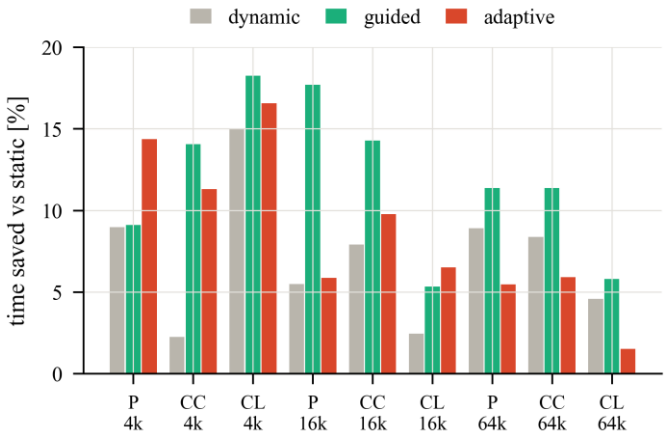


Fig. 4. Run time saved relative to static at 8 threads for the clustered inputs at N = 4,096, 16,384 and 65,536 (4k, 16k, 64k). P: Plummer sphere; CC: colliding clusters; CL: cold collapse.

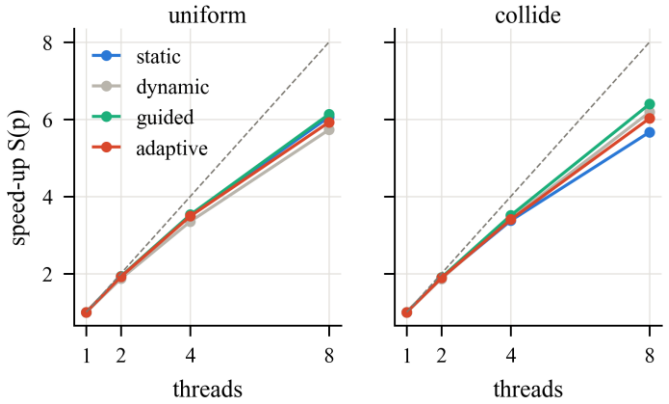


Fig. 5. Speed-up versus threads at N = 65,536 for the balanced uniform input and the clustered colliding-cluster input. Dashed: ideal.

D. Controller Sensitivity and Overhead

Table IV varies the controller on the cold collapse (N = 65,536, 8 threads). Checking every 20 steps reacts late to the collapse, and a high threshold of 0.10 switches on noise five times; k = 1–5 with a high threshold of 0.15–0.25 works best. For dynamic scheduling on the colliding clusters, chunk 1 (10.80 s) was slower than static (10.43 s), while chunks of 8, 64 and 512 took 9.56, 9.33 and 9.35 s. On the smallest uniform problem (N = 1,024, 400 steps), the controller took 0.128 s against 0.120 s for static and 0.139 s for dynamic with chunk 1.

TABLE IV
CONTROLLER SENSITIVITY, COLD COLLAPSE

Setting	Time [s]	Switches
k = 1	6.34	7
k = 5 (default)	6.34	1
k = 10	6.51	1
k = 20	7.13	1
$\lambda_{high} = 0.10$	7.37	5
$\lambda_{high} = 0.25$	6.06	1
$\lambda_{high} = 0.40$	6.76	1

E. Hybrid MPI × OpenMP

TABLE V
MPI × OPENMP SPEED-UP, N = 65,536

Ranks × threads	Uniform (best)	Collide static	Collide adaptive	Rank λ
2 × 1	1.32	1.26	–	0.019
4 × 1	2.59	2.50	–	0.084
1 × 8	5.69	4.30	5.26	0.000
2 × 4	5.56	4.56	4.78	0.036
4 × 2	5.62	4.48	4.79	0.107
2 × 8	6.99	5.52	6.38	0.022
4 × 4	7.04	5.50	5.85	0.066

Table V reports speed-ups against the single-thread baseline. All configurations produce the same final state. With the same eight cores, 1 rank × 8 threads is the fastest split: every additional rank repeats the sort and tree build and exchanges data every step. The controller improves on static in every configuration on the clustered input, saving 5–18 %. The imbalance between ranks reached λ = 0.11, which the OpenMP controller cannot correct. With both hardware threads of the eight performance cores (16 workers) the speed-up reaches 7.0×.

F. GPU Acceleration

At N = 65,536 the single-precision tiled kernel takes 24.0 ms per step against 2117 ms for 8 CPU threads, a factor of 88, with an rms error of 4.6×10^{-6} (Fig. 6); double precision is limited by the low FP64 throughput of the laptop GPU. The Barnes–Hut hybrid with 256-body leaves is 1.39, 1.30, 1.29× faster than the best CPU-only configuration at N = 4,096, 16,384 and 65,536; with small leaves the CPU tree walk dominates and the GPU part does not pay off.

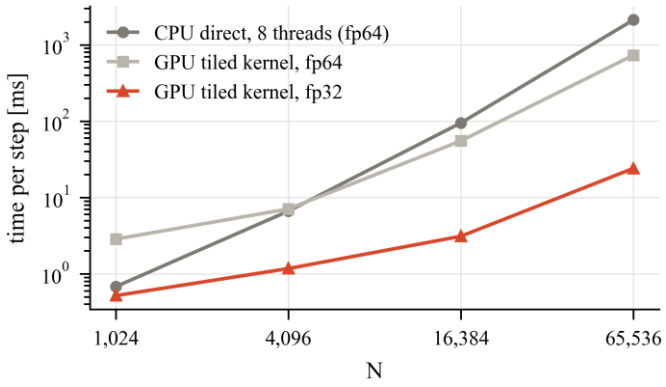


Fig. 6. Direct summation, time per step versus N: CPU with 8 threads and the tiled CUDA kernel in double and single precision.

VII. Discussion and Limitations

The controller removes the penalty of static scheduling on clustered inputs without requiring a schedule to be chosen by hand, but on this platform guided scheduling was equally good or better. The controller spends the balanced early phase of a run in static before it detects imbalance, and guided scheduling keeps large contiguous Morton blocks for its first chunks, preserving cache locality, while its small final chunks absorb the imbalance.

A time-based trigger can also react to noise: on the uniform input with 1 rank × 8 threads, the controller switched to dynamic on a timing spike and was 17 % slower than static. Imbalance between MPI ranks is outside the controller's reach, and all measurements come from a single laptop, where ranks cost more than threads and timings depend on temperature and turbo state. The replicated tree also limits memory scaling for very large N.

VIII. Future Work

Three extensions have been implemented and measured on a second machine (AMD Ryzen 7 7730U): cost-weighted rank partitioning, which cuts the Morton order into pieces of equal measured work and was up to 17.2 % faster with 4 ranks on colliding clusters; a parallel Morton sort and tree build, 2.07–2.53× faster at 8 threads; and an automatic mode in which the controller tries dynamic(8), dynamic(64) and guided(8) for one window each and keeps the fastest. Remaining work is to let the controller tune the chunk size, to distribute the tree instead of replicating it, and to evaluate on a multi-node cluster with GPUs on every node.

IX. Conclusion

We built and validated a hybrid MPI–OpenMP–CUDA Barnes–Hut simulator whose results are bit-identical across all parallel configurations, and a run-time schedule controller that uses only the standard OpenMP interface. As bodies cluster, static scheduling leaves the thread that owns the dense region with up to 3.7× the work of the lightest thread. The controller matches static scheduling on balanced input and beats it in all nine clustered configurations, by up to 16.6 %, but does not outperform the best fixed schedule, guided, on this hardware. The single-precision CUDA kernel is 88× faster than 8 CPU threads for direct summation.

References

[1] J. Barnes and P. Hut, "A hierarchical O(N log N) force-calculation algorithm," *Nature*, vol. 324, pp. 446–449, 1986.

[2] J. P. Singh, C. Holt, T. Totsuka, A. Gupta, and J. L. Hennessy, "Load balancing and data locality in adaptive hierarchical N-body methods," *J. Parallel Distrib. Comput.*, vol. 27, no. 2, pp. 118–141, 1995.

[3] L. Nyland, M. Harris, and J. Prins, "Fast N-body simulation with CUDA," in *GPU Gems 3*, Addison-Wesley, 2007, ch. 31.

[4] M. Burtscher and K. Pingali, "An efficient CUDA implementation of the tree-based Barnes Hut n-body algorithm," in *GPU Computing Gems Emerald Ed.*, Morgan Kaufmann, 2011, pp. 75–92.

[5] S. Keller, A. Cavelan, R. Cabezón, L. Mayer, and F. M. Ciorba, "Cornerstone: Octree construction algorithms for scalable particle simulations," in *Proc. Platform for Advanced Scientific Computing Conf. (PASC)*, 2023.

[6] R. Cazalbou, F. Duchaine, E. Quémerais, B. Andrieu, G. Staffelbach, and B. Maugars, "Hybrid multi-GPU distributed octrees construction for massively parallel code coupling applications," in *Proc. PASC*, 2024, pp. 14:1–14:11.

[7] V. Nagarajan, R. Gangaraju, K. Sundararajah, A. Pelenitsyn, and M. Kulkarni, "RT–BarnesHut: Accelerating Barnes–Hut using ray-tracing hardware," in *Proc. ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP)*, 2025.

[8] J. H. Müller Korndörfer, A. Eleliemy, A. Mohammed, and F. M. Ciorba, "LB4OMP: A dynamic load balancing library for multithreaded applications," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 4, pp. 830–841, 2022.

[9] A. Mohammed, J. H. Müller Korndörfer, A. Eleliemy, and F. M. Ciorba, "Automated scheduling algorithm selection and chunk parameter calculation in OpenMP," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, pp. 4383–4394, 2022.

[10] F. M. Ciorba, A. Mohammed, J. H. Müller Korndörfer, and A. Eleliemy, "Automated scheduling algorithm selection in OpenMP," in *Proc. 22nd Int. Symp. Parallel Distrib. Comput. (ISPDC)*, 2023, pp. 106–109.

[11] J. H. Müller Korndörfer, A. Mohammed, A. Eleliemy, Q. Guilloteau, R. Krummenacher, and F. M. Ciorba, "A comparative study of OpenMP scheduling algorithm selection strategies," arXiv:2507.20312, 2025.

[12] F. H. S. da Silva, J. B. Fernandes, I. M. Sardina, T. Barros, S. X. de Souza, and I. A. Assis, "Auto-tuning for OpenMP dynamic scheduling applied to full waveform inversion," *Comput. Geosci.*, vol. 202, art. 105932, 2025.

[13] F. A. Gratl, S. Seckler, H.-J. Bungartz, and P. Neumann, "N ways to simulate short-range particle systems: Automated algorithm selection with the node-level library AutoPas," *Comput. Phys. Commun.*, vol. 273, art. 108262, 2022.

[14] S. J. Newcome, F. A. Gratl, P. Neumann, and H.-J. Bungartz, "Towards auto-tuning multi-site molecular dynamics simulations with AutoPas," *J. Comput. Appl. Math.*, vol. 433, art. 115278, 2023.

[15] S. J. Newcome, F. A. Gratl, M. Lerchner, A. Pazar, M. K. Mishra, and H.-J. Bungartz, "Algorithm selection in short-range molecular dynamics simulations," in *Proc. Int. Conf. Computational Science (ICCS)*, 2025.